

Agent QA de création de bugs pour VS Code

BugIt

Transformez des notes de test sommaires en tickets de bug soignés et vérifiés — créés dans votre outil de suivi en quelques secondes.

Un agent QA qui s'adapte au workflow de votre équipe.

Conçu par un ingénieur QA senior qui rédige et relit des rapports de bug chaque jour.

🔒 Privé par conception — votre travail n'est jamais envoyé à Taskivator.

~10 min

économisées par bug

**Anti-
doublons**

avant la création

~15 min

configuration unique

\$39.99/an

accès anticipé · puis \$59.99

POUR COMMENCER

Glossaire en 60 secondes

Aucune compétence technique requise — les seuls mots qui comptent, en termes simples.

VS Code

Une application gratuite de Microsoft — la « fenêtre » dans laquelle vit cet agent.

GitHub Copilot

Le cerveau de l'IA qui rédige les rapports. La seule chose que vous payez.

BugIt

Votre assistant dans cette fenêtre. Discutez avec lui comme avec un collègue.

Outil de suivi

Là où votre équipe conserve ses bugs (Jira, ADO, GitHub...).

Serveur MCP

Un petit « pont » vers votre outil de suivi. Activez-le une fois.

Jeton

Un mot de passe privé utilisé par le pont. Stocké en lieu sûr.

config.json

Une fiche de paramètres. L'agent la remplit pour vous.

Chat

Là où vous tapez : « Write a bug report: l'appli plante à l'enregistrement ».

Si vous savez installer une application, vous connecter et taper une phrase, vous savez utiliser cet outil.

COÛT ET LICENCE

Combien ça coûte

Compatible avec votre modèle d'IA préféré. BugIt pilote le modèle à l'intérieur de GitHub Copilot (GPT, Claude ou Gemini) — ou fonctionne en mode autonome avec votre propre clé.

Gratuit

L'offre Copilot Free couvre un usage occasionnel.

~\$10/mois

Copilot Pro pour un usage quotidien.

Aucune clé API

Aucun hébergement, aucun frais par bug.

Pas de Copilot ?

Apportez votre propre clé OpenAI/Anthropic.

\$39.99/an prix d'accès anticipé, puis \$59.99/an — licence annuelle, un appareil à la fois, toutes les fonctionnalités. **Équipes** : 5 sièges \$199/an en accès anticipé, puis \$249.99/an.

Ça revient à moins de **\$3.50 par mois** — moins de deux mois de ChatGPT Plus, mais pour une année entière. Les outils QA concurrents facturent leur prix **tous les mois** ; économisez le temps d'une **poignée de bugs** et c'est déjà rentabilisé.

Licence Équipe

une seule commande partage votre configuration d'outil de suivi, votre glossaire et votre style de ticket avec les 5 sièges — `share.py export`, les coéquipiers font `import` — pour que tout le monde crée des tickets exactement au même format dès le premier jour.

POURQUOI ÇA EN VAUT LA PEINE

Ce que vous obtenez réellement

Pas « une IA qui rédige des bugs » — un agent QA qui s'intègre à la façon dont votre équipe travaille déjà.

Sans BugIt	Avec BugIt
Rédiger chaque bug à la main	Une phrase → un ticket complet et vérifié
Les doublons s'accumulent sur le tableau	Anti-doublons automatique avant la création, même en cas de formulation différente
Le format varie selon chaque testeur	Un modèle cohérent, à chaque fois
Traduire et consulter les specs à la main	Multilingue, et le contexte est récupéré pour vous

Compatible avec l'outil de suivi que votre équipe utilise déjà

BugIt rédige le ticket pour n'importe quel outil de suivi et le crée via le connecteur que vous activez — changez à tout moment.

Configuration assistée, mappage de champs testé — gravité/priorité définie automatiquement :

Jira Cloud Azure DevOps

Source de connaissances assistée (Atlassian RoVo MCP) :

Confluence Cloud

Expérimental — un point de terminaison connu que BugIt configure et vérifie en direct sur votre compte :

Sentry GitHub Linear Notion

Accompagnement à la configuration uniquement — via votre propre serveur MCP compatible :

GitLab Shortcut YouTrack Bugzilla ClickUp Asana Trello Crashlytics BugSnag TestRail
Xray Slack Teams

Non pris en charge par la configuration automatisée :

S3 Google Drive Azure Blob

Il vérifie aussi les doublons, recherche les crashes, récupère le contexte des specs, traduit entre langues, et rédige les commentaires de vérification/clôture.

CONFIGURATION

Opérationnel en 6 étapes

Configuration unique dans VS Code. L'agent installe ce qui manque.

- 1 Installez VS Code**
Téléchargez-le depuis code.visualstudio.com et ouvrez-le.
- 2 Installez GitHub Copilot**
Extensions → recherchez « GitHub Copilot » → Install → connectez-vous.
- 3 Ouvrez ce dossier**
File → Open Folder. L'agent se charge depuis `.github/agents/`.
- 4 Sélectionnez l'agent**
Dans le chat, basculez le menu déroulant de mode vers **BugIt QA Agent**.
- 5 Tapez « Begin setup »**
Répondez à quelques questions ; l'agent écrit `config.json`. En cas d'interruption, le retaper reprend là où c'était plutôt que de tout recommencer.
- 6 Démarrez les serveurs MCP, dites « hi »**
Activez vos ponts — vous êtes prêt à créer des tickets. Par défaut, votre clé de licence passe par une invite de terminal masquée, jamais par le journal du chat.

Raccourcis : `preset.py game` pour partir d'un modèle · `setup_wizard.py` sans Copilot · `doctor.py` pour une liste de vérification de l'état de préparation.

Un manuel complet, étape par étape, est inclus.

Après votre achat, vous recevrez un guide de configuration et d'utilisation détaillé, clic par clic — en français — écrit pour des personnes qui n'ont jamais fait ça, afin que la configuration soit simple et que vous ne restiez jamais bloqué.

AVANT D'ACHETER — CE QU'IL VOUS FAUT

Prérequis	Pourquoi
VS Code (gratuit)	L'agent s'exécute à l'intérieur
GitHub Copilot (recommandé)	Alimente l'agent — ou apportez votre propre clé OpenAI/Anthropic
Python 3.10–3.13	Exécute les outils utilitaires
Un compte sur un outil de suivi	Là où vont les bugs — configuration assistée avec mappage de champs testé pour Jira et Azure DevOps (Azure DevOps via la préversion publique du MCP distant de Microsoft). GitHub et Linear sont expérimentaux et vérifiés en direct sur votre compte ; tous les autres outils de suivi utilisent votre propre serveur MCP compatible

Bon à savoir

BugIt s'exécute dans **VS Code** (pas dans d'autres applications de chat) ; il pilote **votre** modèle d'IA plutôt que d'en fournir un ; le mappage de champs testé (gravité/priorité définie automatiquement) concerne **Jira + Azure DevOps** (Azure DevOps via la préversion publique du MCP distant de Microsoft), Confluence utilise la même connexion Atlassian assistée, Sentry, GitHub, Linear et Notion sont expérimentaux et vérifiés en direct sur votre compte, et tout le reste utilise votre propre serveur MCP compatible ; et le masquage des données ainsi que les autres protections sont des aides, pas des garanties — **vous relisez et approuvez chaque ticket**, et les résultats sont plus constants avec un modèle d'IA plus puissant qu'avec un modèle très léger/gratuit.

EN ACTION

Une phrase en entrée. Un ticket propre en sortie.

C'est tout l'intérêt de l'outil — tapez-la comme vous la diriez à un collègue ; l'agent fait le reste.

YOU TYPE

"the app crashes every time I tap Save on iPhone"



BUGIT RÉDIGE LE BROUILLON — VOUS RELISEZ, PUIS VOUS DITES « YES »

[Crash] L'appli se ferme en appuyant sur Enregistrer (iPhone)

Étapes	1. Ouvrir l'appli 2. Effectuer une modification 3. Appuyer sur Enregistrer
Attendu	La modification est enregistrée et l'appli reste ouverte
Constaté	L'appli se ferme instantanément — à chaque fois
Détails	Reproductibilité 5/5 · Gravité Critique · Plateforme iOS (iPhone)

Puis il vérifie les doublons et crée le ticket dans votre outil de suivi après votre « yes » — en définissant le champ de priorité propre à l'outil de suivi (pas une valeur générique par défaut) — et vous transmet le lien. Envie d'ajuster avant ? Dites simplement « *make severity high* ». Envie de vous entraîner ? Dites `dry run` et il rédige tout sans rien créer.

AU QUOTIDIEN, VOUS TAPEZ SIMPLEMENT

Vous tapez	Vous obtenez
<code>Write a bug report: ...</code>	Rapport complet, vérification des doublons, détection automatique de version
<code>Close #ID</code> · <code>Translate to ja: ...</code>	Commentaire de vérification/clôture · traduction cohérente avec le glossaire
<code>Update</code> · <code>Back up my settings</code> · <code>help</code>	Mise à jour signée · instantané local · liste complète des commandes

EXTRAS FACULTATIFS

Les fonctionnalités avancées expliquées

Activez-les dans `Begin setup`, puis déclenchez-les depuis le chat.

Triage digest

Tapez `Triage digest` pour un résumé de standup instantané — ce qui a été créé/modifié, le décompte par gravité et par catégorie.

Export bug to file

`Export bug to file` enregistre le rapport en Markdown/JSON local au lieu de le créer — idéal hors ligne ou pour garder une copie.

Undo last filing

`Undo last filing` rappelle le dernier ticket ou commentaire créé par l'agent et, après votre confirmation, le supprime ou le clôture partout où votre outil de suivi le permet.

Alertes SLA

Les bugs de gravité élevée reçoivent un marqueur SLA/échéance dès leur création, pour que les problèmes urgents ressortent automatiquement.

Champs par catégorie

Chaque catégorie ajoute ses propres champs (crash → pile d'appels, UI → capture d'écran) pour que chaque rapport capture le bon niveau de détail.

Tout est facultatif. Ignorez ces fonctionnalités et la boucle principale rédiger → relire → créer fonctionne parfaitement.

CONFIANCE

Sécurité et confidentialité intégrées

Ce ne sont pas des options — elles sont actives par défaut, et c'est pour ça que les équipes font confiance à Buglt avec de vrais tickets.

Nous ne voyons jamais votre travail

Buglt s'exécute dans votre éditeur ; vos tickets ne sont transmis qu'au modèle d'IA et à l'outil de suivi que vous connectez. La seule chose qui nous parvient, ce sont des données de licence/mise à jour — votre clé, un identifiant d'appareil haché à sens unique, et (seulement si vous en avez défini un lors de la configuration) le libellé de siège que vous avez choisi pour distinguer vos appareils. Aucune télémétrie, aucune analyse, jamais.

Sauvegardes automatiques

Avant chaque mise à jour — et à tout moment sur demande — votre config, votre glossaire, votre style maison, votre licence et vos connexions sont capturés sur votre appareil. Une seule ligne les restaure : `Restore my settings`.

Rien n'est créé sans votre « oui »

Chaque ticket et commentaire est présenté en aperçu ; les créations irréversibles exigent une confirmation saisie au clavier. Dites `dry run` pour un mode d'entraînement sans aucune écriture.

Mises à jour signées et vérifiées

Les mises à jour sont signées cryptographiquement avec une clé conservée *séparément* de la licence. L'agent vérifie chaque octet et refuse tout élément altéré — puis conserve tous vos paramètres.

Aucun secret sur le disque

Les jetons résident dans le gestionnaire d'identifiants de votre système, jamais dans un fichier. `config.json` ne contient que des organisations et des URL — et un validateur signale tout ce qui ressemble à un secret divulgué avant même son enregistrement, que ce soit dans la config ou dans une commande exécutée par l'agent pour vous.

Résistant à l'injection de prompt

Le texte extrait de pages ou de tickets est traité comme une donnée, jamais comme une commande — les instructions injectées sont signalées, jamais exécutées, et portées à votre attention.

Personnalisez en toute sécurité

`config.json`, le style maison, le glossaire et la configuration MCP sont à vous — conservés à chaque mise à jour. Le fichier de l'agent et les autres fichiers de `.github/instructions/` ne le sont pas : les modifier à la main les fait écraser silencieusement, sans sauvegarde permettant de les restaurer.

Privé par conception. Vos rapports de bug, vos specs, votre glossaire et vos paramètres restent les vôtres. L'agent ne parle qu'aux outils que vous avez connectés et à votre propre modèle d'IA — jamais à nous.

Une licence qui vous respecte : vérifiée hors ligne, et elle échoue en mode **ouvert** — une panne du serveur de licence ne vous bloque jamais dans votre travail. Un appareil à la fois ; changez d'appareil librement.

ASSISTANCE

Dépannage

Symptôme	Solution
« No tracker enabled »	Lancez <code>Begin setup</code> , choisissez un outil de suivi
Le serveur affiche 	Ouvrez <code>.vscode/mcp.json</code> , cliquez sur Start ; ou dites « fix servers »
Demande une reconnexion	« show my tokens » (affiché localement), collez-le dans l'invite
Refuse de créer le ticket	Vérifiez <code>validate_config.py --test</code> ; confirmez l'outil de suivi principal + la connexion
Semble spécifique aux jeux	Modifiez <code>config.categories</code> selon votre domaine

Toujours bloqué ? Rendez-vous sur **bugit.dev** ou écrivez à **support@bugit.dev** (support assuré en anglais) — une vraie personne vous répondra. Souci de configuration dans vos 7 premiers jours ? Nous vous aiderons à démarrer, ou à obtenir un remboursement via la boutique.

Une année entière pour moins de deux mois de ChatGPT Plus

\$39.99/an, c'est moins de **\$3.50 par mois**, et le compteur démarre à l'activation — aucun frais par bug. Les outils QA concurrents facturent leur prix **tous les mois**. Économisez le temps d'une poignée de bugs et c'est déjà rentabilisé.

~10 min économisées/bug

une ligne → un ticket complet

Compatible avec votre outil de suivi

celui que vous utilisez déjà

Anti-doublons

avant la création

Multilingue

10+ langues, verrouillé par le glossaire

*J'ai créé **BugIt** parce qu'après des années à rédiger et relire des rapports de bug chaque jour, j'en avais assez de passer mon temps sur une mise en forme répétitive plutôt qu'à trouver de vrais problèmes. BugIt est l'outil que j'aurais aimé avoir.*

Langues : en, ja, fr, de, es, pt-br, it, ko, zh, ru & plus · Solo \$39.99/an en accès anticipé (puis \$59.99) · Équipe 5 sièges \$199/an en accès anticipé (puis \$249.99)

BugIt · mises à jour gratuites au sein de votre version majeure tant que votre licence est active · un achat = un utilisateur (aucune revente ni redistribution)

© 2026 Taskivator. All Rights Reserved. · bugit.dev